Chapter 7: Retrieval-Augmented Generation

7.1 Why standard LLMs fall short

One technique in AI deployment addresses a core limitation of large language models: they often lack the specialized knowledge needed in particular domains. For particular, you are most likely to get a hallucinated answer if you ask an LLM about a niche fact, e.g., some particular aspect of New York City housing regulations, that does not appear frequently in the training data.

One way to solve this is to upload documents containing the specialized knowledge to the LLM when you query it. If your knowledge base is small, such as a two-page policy, you can paste the whole text into the prompt, so that the model reads it along with your question.

That stops working however when the knowledge base is too large to fit in the model's context window (the context window, discussed in the last chapter, is the information that that model bases its responses on). A context window of a million tokens, which is typical of today's frontier models, holds roughly 750,000 words, and a company's documents, a library of research papers, or a city's legal code can easily exceed that.

Even when everything does fit in the context window, sending all of it to the LLM with every question is slow and costly, and models can overlook details buried in very long inputs.

Second, even for topics that are well covered within its training data, the model cannot tell you how it arrived at a particular answer. In fact, the answer is drawn from a statistical blend of everything it has ever read. As a result, it is hard to confirm that the LLM's response is correct.

7.2 How RAG works

RAG stands for Retrieval-Augmented Generation. It is useful when you have a large collection of source documents that should inform answers to a query — for example, the

Copyright © 2026 Andrew Dessler

full corpus of New York City housing regulations used to answer questions about tenants' rights.

A RAG system works in two stages (Figure 7.1). The first stage happens once, before anyone asks a question. The system takes the source documents — the files you want the model to draw on, such as a company's policies, a set of academic papers, or a city's housing law — and splits each one into passages of a few paragraphs. It then builds an index that records what each passage is about. The indexed documents together form the system's knowledge base.

The second stage repeats every time someone enters a question. When you enter a query, the system searches the knowledge base for passages that match the query most closely. It then sends the query, together with only those passages from the knowledge base, to the LLM. The LLM then responds based on all of that.

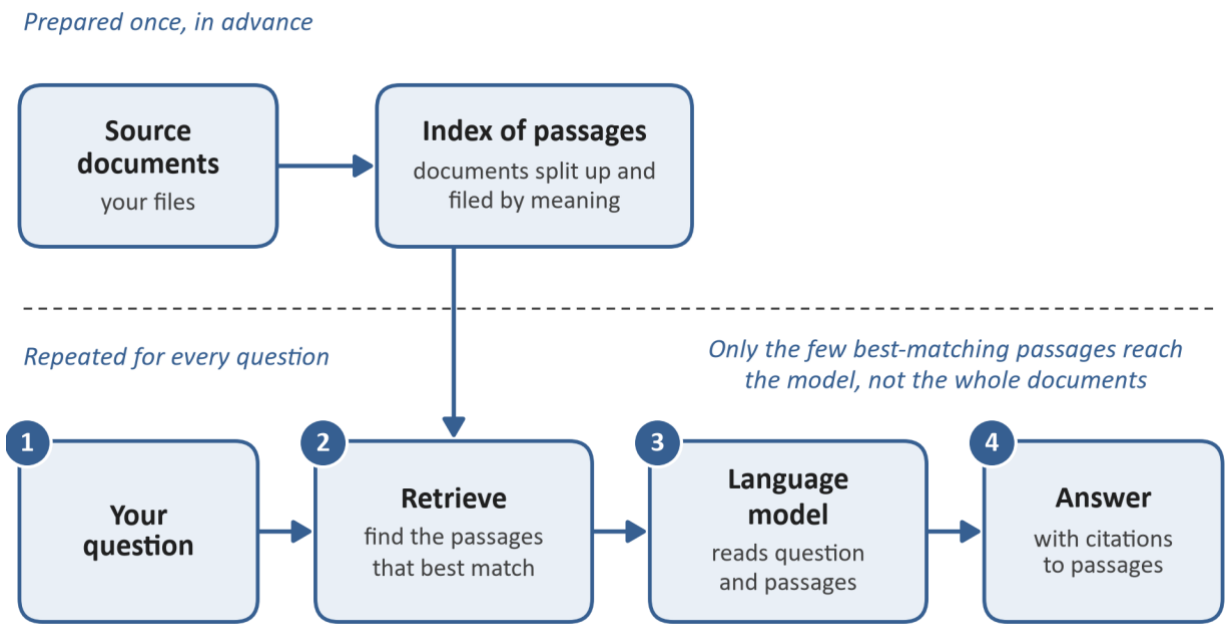


Figure 7.1. How a RAG system answers a question. The documents are prepared once, in advance; each question then retrieves only the best-matching passages, so the documents never have to fit in the model's context window.

The advantage of RAG is that the LLM does not need to see the whole knowledge base. Only the few matching passages travel with the question, so the documents do not have to fit in the model's context window.

Copyright © 2026 Andrew Dessler

A RAG system can therefore utilize far more material than any context window could hold, such as hundreds of research papers or an entire policy manual, because retrieval acts as a filter that picks out the small part the model needs for this particular question. The LLM should also respond faster and (if you're paying by the token) it should be cheaper.

Finally, the model can point to the passages that it used to derive the answer. This allows the human user to verify the results and fundamentally reduces the chance of hallucination.

Gemini Notebook is a commercial version of a RAG system. A researcher working with 500 academic papers can upload them to the platform and ask targeted questions, receive cited summaries, and then read the specific passages most relevant to their work. The AI handles retrieval and initial synthesis; the human handles judgment and interpretation.

7.3 Key takeaways

- RAG greatly reduces the hallucination problem for specific domains by grounding model answers in user-provided documents and providing verifiable citations.
- Gemini Notebook is a leading consumer example of RAG: every answer is cited, every citation is clickable, and the model cannot invent information outside your document set.

Copyright © 2026 Andrew Dessler